

M4 — Arbiter / sequential-impulse solver ⌚

Status: planned. This is the load-bearing 80% — where "feel" lives.

Goal

Reproduce Nape's contact solver bit-exactly: the per-iteration sequential impulse loop, warm-starting, restitution and friction models, and the `normalImpulse` accumulation the game reads for breakable crates.

Scope

- Velocity-constraint solve over `velIters` iterations (the game uses 10).
- Position/penetration solve over `posIters` iterations.
- **Warm-starting:** persisting accumulated impulses across frames keyed by contact id (depends on M3 ids).
- Restitution (elasticity combine), static/dynamic friction, rolling friction.
- `CollisionArbiter.normalImpulse` exposure (breakable-crate impact magnitude).

Gate

- A ball dropped onto a static floor settles bit-identically over many steps (resting contact + restitution + friction all exercised).
- The **level-9 bank shot** scenario — the project's known friction/feel litmus — is bit-identical end-to-end.
- `normalImpulse` reported on contact matches the reference bit-for-bit.

Risks / open questions

- Iteration order over arbiters/contacts changes float sums — must match the reference's arbiter ordering (ties into M2/M3 ordering).
- Warm-start lookup/reset semantics across frames are subtle and easy to get *almost* right; the multi-step gate is what catches it.